JIG

Inventory Management System

Project Overview

Overview

Current Jig Department operations rely on manual tracking of specialized tools and materials, leading to persistent stock inaccuracies and restricted data accessibility for staff. JIG IMS is a web-based Inventory Management System (IMS) designed to centralize inventory data and support dynamic database schema updates. By providing a user-friendly interface, this system will digitize department workflows and ensure real-time oversight of all specialized assets.

The Solution

SwiftStock leverages IoT barcode integration and an automated reorder engine to eliminate manual entry errors. By transitioning to a centralized database, the system provides a "single source of truth" accessible across the warehouse, sales, and procurement departments.

Key Capabilities (to improve)

- **Real-Time Tracking:** Instant updates on stock levels via mobile QR Code scanning and updating in inventory.
- **Automated Procurement:** Predictive logic that generates purchase request forms when an item hits minimum quantity.
- **Role-Based Security:** Tiered access to ensure data integrity and financial privacy.

Technical Documentation

Flow Chart (on going)

[insert flowchart here]

Tech Stack

This is a practical, high-performance choice for internal department tools where rapid deployment and dynamic data handling are the top priorities.

- **Frontend:** HTML5, CSS3, and JavaScript (Standard web technologies used to build a responsive, user-friendly interface).
- **Backend:** PHP (Server-side logic featuring a dynamic database handler to adapt to schema changes automatically).
- **Database:** MySQL via XAMPP (A reliable relational database used for centralized storage of specialized tool and material records).
- **Synchronization:** AJAX (Enables 'Sync Changes' functionality for seamless bulk updates without requiring a full page reload).

Best for: Small-to-medium departments, rapid internal tool development, and environments requiring flexible, evolving database structures.

System Architecture

The system follows a **WAMP** architecture (Windows, Apache, MySQL, PHP) implemented through the **XAMPP** stack.

- **Frontend:** Dynamic UI rendering using PHP to automatically reflect current database column names and structures.
- **Backend:** PHP environment featuring a dynamic database handler that adapts to schema changes without manual code updates.
- **Database:** MySQL (**ims_db**) for centralized storage, integrated via a unified configuration file for consistent connectivity.
- **Synchronization:** AJAX-enabled 'Sync Changes' feature for seamless bulk updates through a modal interface without page reloads.

Database Schema (on going)

The system utilizes four primary collections. Below is the structure for the `Product` and `Transaction` models:

***Product Model*:** | Field | Type | Description | | :--- | :--- | :--- |

Installation and Deployment

User Manual

User Roles & Permissions

The Inventory Management System provides tailored access levels to streamline staff workflows while ensuring that sensitive administrative features—such as database schema updates—remain restricted to authorized users.

Administrator (Full Access):

- **System Management:** Full authority over inventory data and schema updates.
- **Data Insights:** Access to all statistics, analytics, and formal reporting.
- **User Management:** Controls the technician roster, including adding and removing user accounts.

Technician (Standard Access):

- **Request Management:** Exclusive access to the 'Components Out' form for requesting specialized tools and materials only accessible through the QR Code.
- **System Restrictions:** Restricted from viewing administrative analytics, inventory editing, or user roster management

Getting Started

Login / Signup Page

As an Administrator. First you will be greeted with the login page

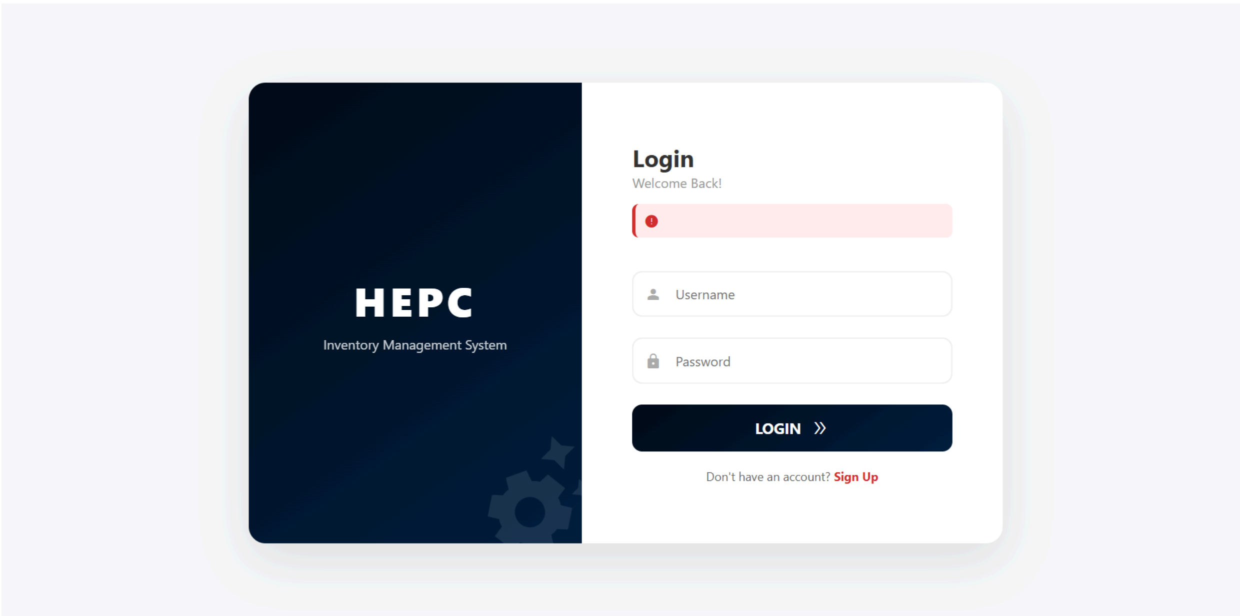


Figure 1.1 Login Page

You can simply write your username and password before logging in. In case you haven't signed up / registered an account yet, you can go ahead and click at the red "Sign Up" text to be redirected

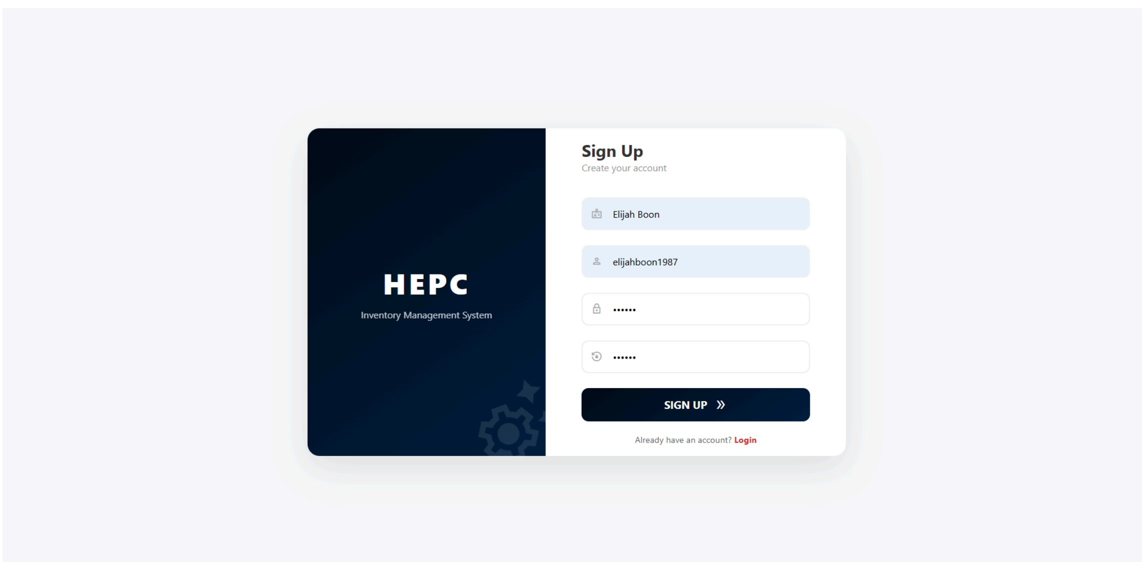


Figure 1.2 Sign Up

After filling up the form clicking the sign up button, you will be automatically logged in and redirected to the IMS dashboard. So be sure to remember the username and password you entered.

Dashboard

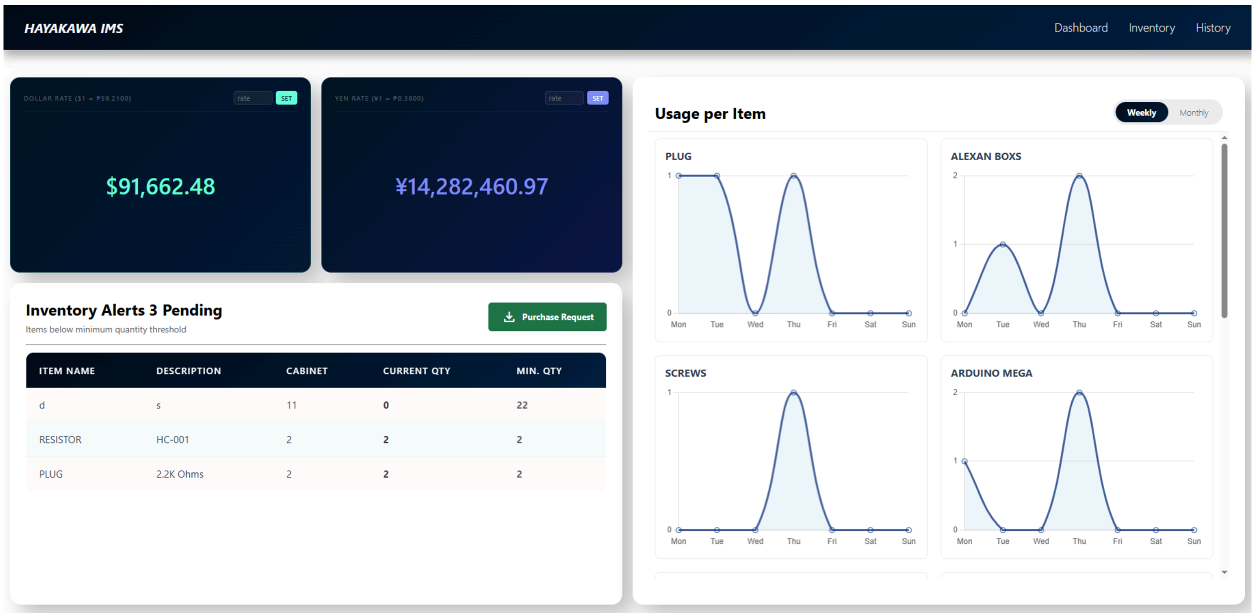


Figure 2.0 Dashboard

We will be walking you through the dashboard and its functionality. First, we have foreign exchange section. Here it displays the current inventory value and converts it from PHP to USD and JPY to be displayed in Dashboard. You can also set a custom Exchange rate for both currencies.

Second, we have inventory alerts below the forex section. Here it displays the items that have reached or exceeded the minimum quantity. You can automatically create a Purchase Request Form in this section for those items that are in risk of going out of stock.

Third, we have the Graph Section which is positioned on the right side. It displays the visualized data of history which are the items that are being pulled out.

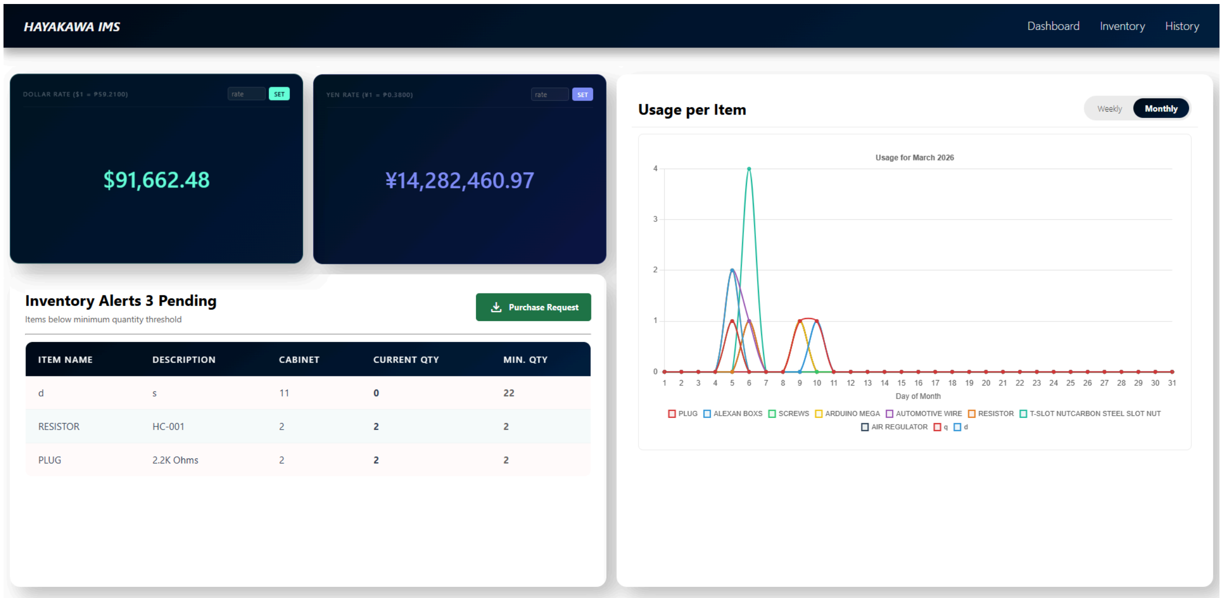


Figure 2.1 Monthly Graph

It shows the pattern of items movement weekly and monthly, helpful for analyzing the consumption patterns.

And lastly we have the navigation bar, which contains the following selection. Inventory, History.

Inventory

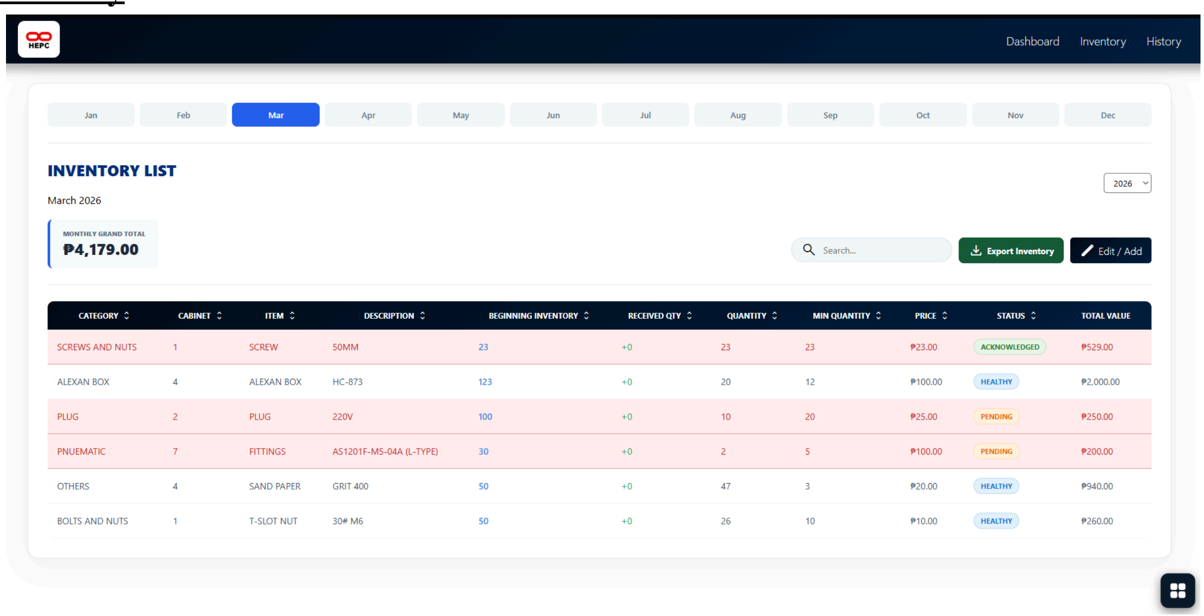


Figure 3.0 Inventory Section

This is your central hub for managing all common materials on hand. To help you get up to speed, here is a quick walkthrough of the interface and the core elements you'll be working with.

Navigating Through Time

Everything starts with the **Month Selector** located at the top of the interface. Its primary function is to let you navigate through monthly and yearly logs, making it easy to backtrack or audit history from a specific period.

The best part? When a month ends, you don't have to re-type everything from scratch. You have the **Carry Over** capability, which allows you to migrate items from the previous month directly to the current one. This keeps your records consistent and saves you a massive amount of encoding time.

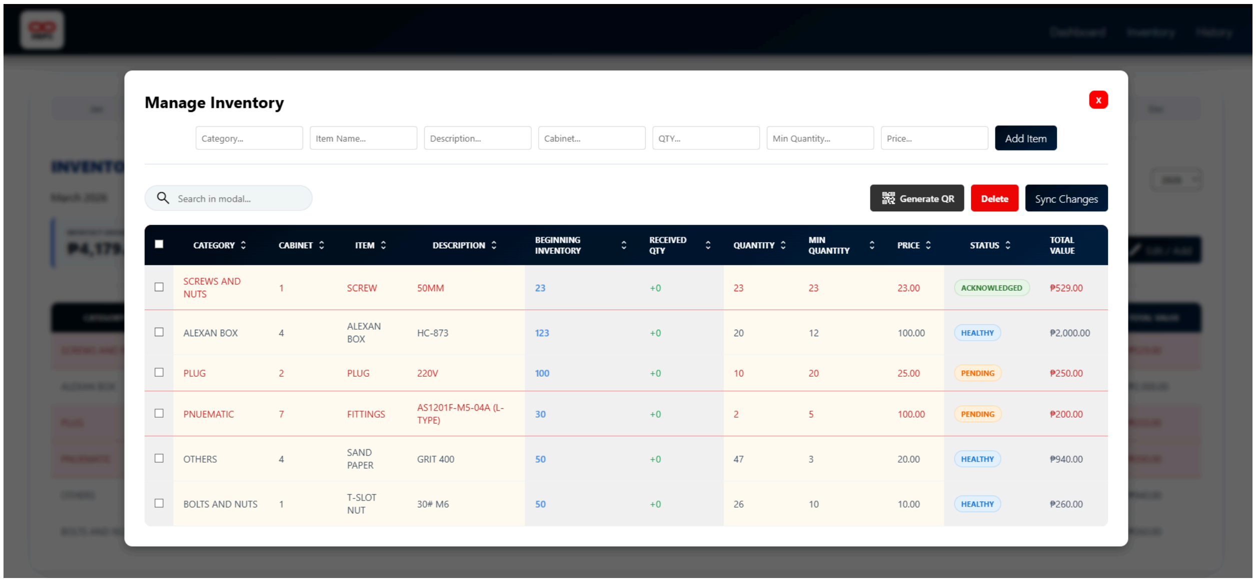
Monitoring & Quick Search

Once you're inside a specific month's record, we've included a **Search Bar** so you can find specific materials in seconds instead of scrolling through long lists. Right next to it, you'll see the **Grand Total**—a real-time display that gives you a quick look at the total monetary value of your current month's inventory in Pesos (PHP).

Core Functionalities

To keep your workflow efficient, we've kept the main controls straightforward with two primary buttons:

- **Export Inventory:** With one click, this instantly generates an **Excel file** of your current monthly list, perfect for reports or external auditing.
- **Edit/Add:** This is your main toolkit. Use this whenever you need to register new materials or update existing quantities in the system.



Inside the Edit/Add Button, you are greeted with a modal pop-up where all the management happens.

Understanding the Inventory Table

To help you get around the system, here's a quick breakdown of how the table works and what each column represents.

Item Identification:

Each entry starts with a **Category** (the general group, like *Electronic Components*) and a **Cabinet** (the physical storage location, like *Cabinet 2*).

The **Item** is the material's name, while the **Description** covers specific technical specs, such as *200 Ohms* for a resistor.

Stock Tracking & Carry-Overs:

The **Beginning Inventory** is crucial for monthly transitions—it logs exactly what was left during the turnover. For example, if you had 10 units left in March, that "10" automatically becomes your April starting point.

Any new stocks added during the month are tracked under **Received QTY**, which calculates the total amount of restocks you've made throughout the month. If you restock 50 units, the system logs it here and continues to accumulate every time you add more..

Meanwhile, the **Quantity** column always reflects the actual, real-time amount you have on hand.

Restocking & Totals:

To avoid running out of supplies, the **Min Quantity** acts as your "red line"—once your stock hits this number, it's flagged for a purchase request.

On the financial side, we have the **Price** (cost per unit) and the **Total Value**, which is simply the price multiplied by your current quantity.

Tracking Item Status

To make monitoring easier, every item is assigned a **Status** so you know exactly what needs your attention:

- **Healthy:** Everything is good! The stock is well above the minimum quantity and no action is needed.
- **Pending:** Heads up—the item has hit or dropped below the minimum quantity and needs immediate restocking.
- **Acknowledged:** The item is resolved in the Inventory Alert , a Purchase Request is already being processed, and you're just waiting for the new stocks to arrive.

Now that you know how to read the inventory logs, **here's how you can actively manage and update the records.** Whether you're adding fresh stock or adjusting existing details, the system is designed to be straightforward:

Adding New Items

In order to add new common items in the inventory, just fill out the form at the top with the required details and hit the **Add Item** button to save it.

Editing Existing Items

For items already in the list, the table itself is interactive:

- **Light Yellow Cells:** These are editable. You can click and type directly to restock quantities, fix typos, set minimum stock levels, or update prices.
- **Gray Cells:** These are locked or read-only values that can't be changed.

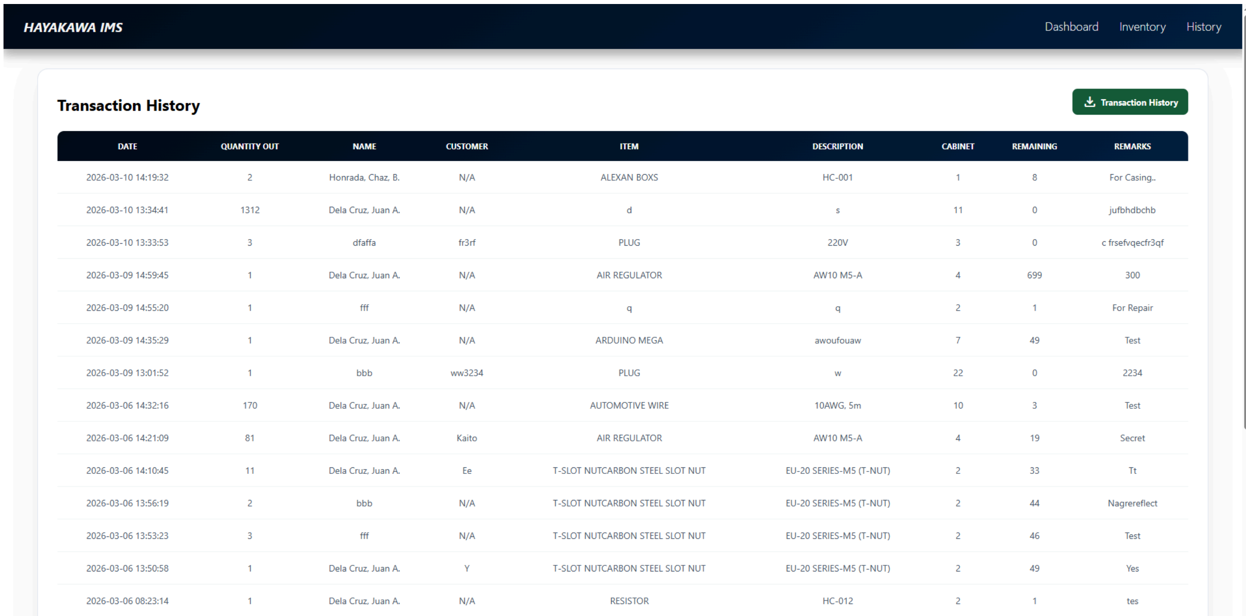
Once you're done with your updates, just hit **Sync Changes** to save everything to the system.

Bulk Actions & Selection Tools

Beyond just viewing data, the table gives you full control over multiple items at once through the **Checkboxes**. These aren't just for show—they unlock two of the most powerful shortcuts in the system:

- **Smart Deletion:** If you need to clean up your list, you can use the checkboxes to selectively delete items or wipe them out in bulk. It saves you the headache of clicking "delete" one by one.
- **QR Code Generation:** This is where it gets efficient. By ticking the checkboxes, you can select specific items (or the entire list) to generate **QR Codes**. Once you've made your selection, just hit the **Generate QR** button. The system will redirect you to a dedicated page with all your QR codes ready for scanning and printing.

.



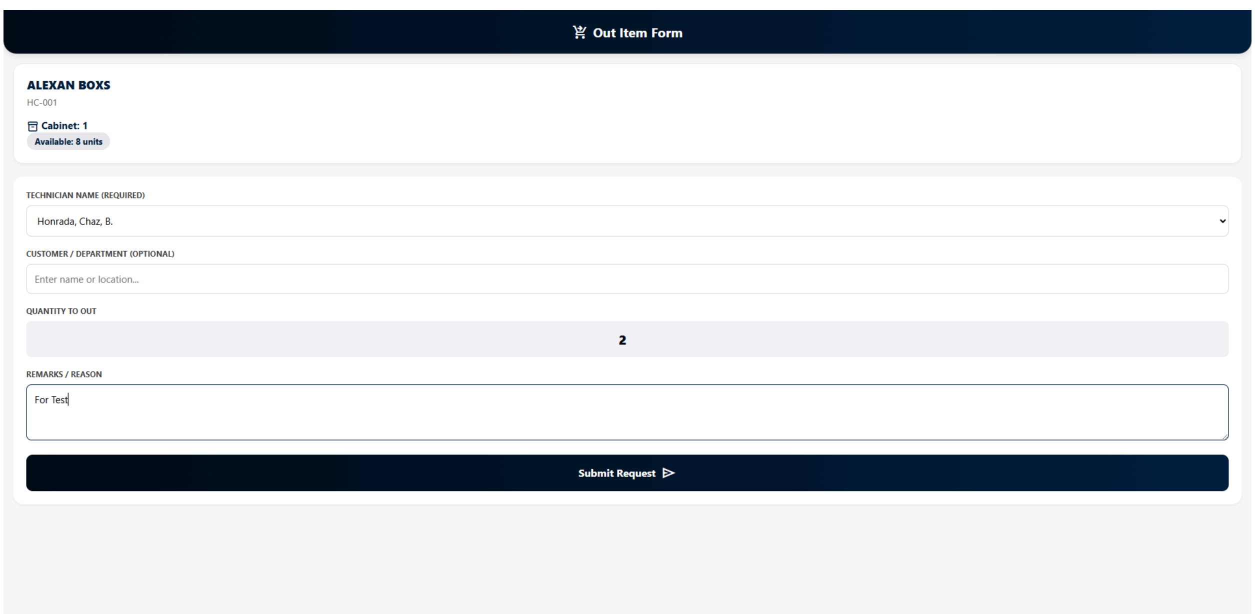
HAYAKAWA IMS

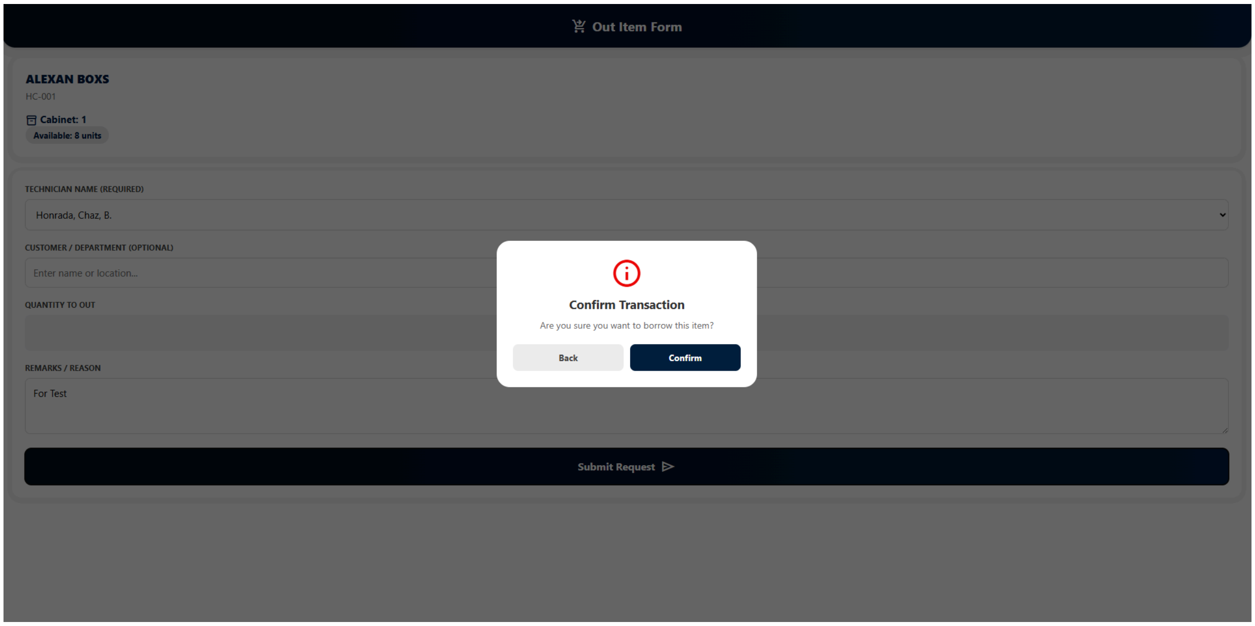
Dashboard Inventory History

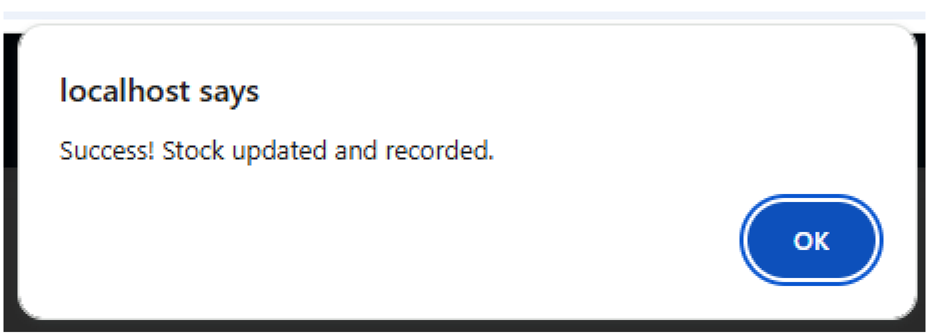
Transaction History

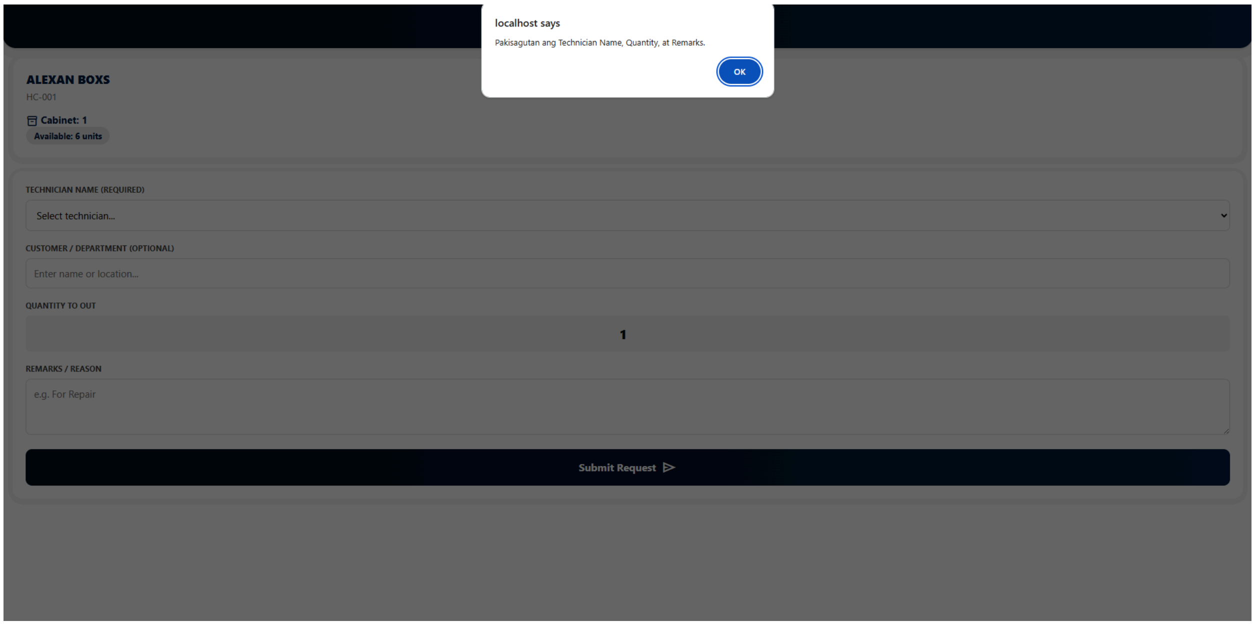
Transaction History

DATE	QUANTITY OUT	NAME	CUSTOMER	ITEM	DESCRIPTION	CABINET	REMAINING	REMARKS
2026-03-10 14:19:32	2	Honrada, Chaz, B.	N/A	ALEXAN BOXS	HC-001	1	8	For Casing..
2026-03-10 13:34:41	1312	Dela Cruz, Juan A.	N/A	d	s	11	0	jufbhdbchb
2026-03-10 13:33:53	3	dfaffa	fr3rf	PLUG	220V	3	0	c frsefvqecfr3qf
2026-03-09 14:59:45	1	Dela Cruz, Juan A.	N/A	AIR REGULATOR	AW10 M5-A	4	699	300
2026-03-09 14:55:20	1	fff	N/A	q	q	2	1	For Repair
2026-03-09 14:35:29	1	Dela Cruz, Juan A.	N/A	ARDUINO MEGA	awoufouaw	7	49	Test
2026-03-09 13:01:52	1	bbb	ww3234	PLUG	w	22	0	2234
2026-03-06 14:32:16	170	Dela Cruz, Juan A.	N/A	AUTOMOTIVE WIRE	10AWG, 5m	10	3	Test
2026-03-06 14:21:09	81	Dela Cruz, Juan A.	Kaito	AIR REGULATOR	AW10 M5-A	4	19	Secret
2026-03-06 14:10:45	11	Dela Cruz, Juan A.	Ee	T-SLOT NUTCARBON STEEL SLOT NUT	EU-20 SERIES-M5 (T-NUT)	2	33	Tt
2026-03-06 13:56:19	2	bbb	N/A	T-SLOT NUTCARBON STEEL SLOT NUT	EU-20 SERIES-M5 (T-NUT)	2	44	Nagrereflect
2026-03-06 13:53:23	3	fff	N/A	T-SLOT NUTCARBON STEEL SLOT NUT	EU-20 SERIES-M5 (T-NUT)	2	46	Test
2026-03-06 13:50:58	1	Dela Cruz, Juan A.	Y	T-SLOT NUTCARBON STEEL SLOT NUT	EU-20 SERIES-M5 (T-NUT)	2	49	Yes
2026-03-06 08:23:14	1	Dela Cruz, Juan A.	N/A	RESISTOR	HC-012	2	1	tes




Out Item Form
ALEXAN BOXS
HC-001
Cabinet: 1
Available: 8 units
TECHNICIAN NAME (REQUIRED)
Honrada, Chaz, B.
CUSTOMER / DEPARTMENT (OPTIONAL)
Enter name or location...
QUANTITY TO OUT
REMARKS / REASON
For Test
Confirm Transaction
Are you sure you want to borrow this item?
Back
Confirm
Submit Request


localhost says
Success! Stock updated and recorded.
OK



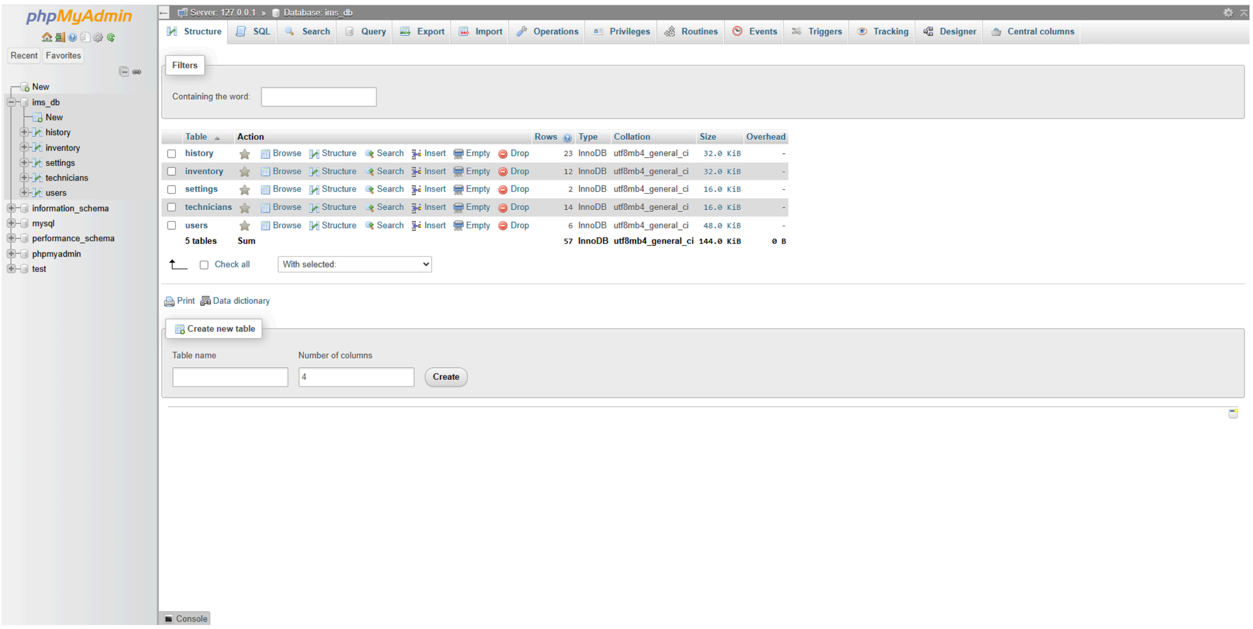
Common Tasks

"How to add a new SKU," "How to generate a monthly report."

Data Backup & Security

How the data is saved and kept safe.

```
└── OJTProject/
    ├── index.php
    ├── script.js
    ├── style.css
    ├── include/
    │   ├── auth_checker.php
    │   └── config.php
    ├── component/
    │   ├── search.js
    │   ├── sort.js
    │   ├── settings/
    │   │   ├── addUser.css
    │   │   ├── addUser.js
    │   │   ├── logout.php
    │   │   ├── settings.css
    │   │   └── settings.php
    │   ├── selectMonth/
    │   │   ├── selectMonth.css
    │   │   └── selectMonth.php
    │   ├── qrGen/
    │   │   ├── qr_gen.html
    │   │   └── qr_gen.php
    │   ├── navbar/
    │   │   ├── nav-bar.css
    │   │   └── nav-bar.php
    │   ├── inventoryTableHandler/
    │   │   ├── inventoryTableHandler.css
    │   │   └── inventoryTableHandler.php
    │   ├── inventoryModal/
    │   │   ├── inventoryModal.css
    │   │   └── inventoryModal.php
    │   ├── inventoryAlertBox/
    │   │   ├── inventoryAlerts.css
    │   │   └── inventoryAlerts.php
    │   ├── graphBox/
    │   │   ├── chart.js
    │   │   ├── graph.css
    │   │   └── graph.php
    │   └── currency/
    │       ├── currency.css
    │       └── currency.php
    └── pages/
        ├── signup/
        │   └── signup.php
        ├── inventory/
        │   ├── inventory.css
        │   ├── inventory.js
        │   ├── inventory.php
        │   └── inventoryFunction.php
        ├── history/
        │   └── history.php
        └── dashBoard/
            ├── dashBoard.css
            └── dashBoard.php
```



Appendix

1. Project Overview & Scope

This section sets the stage. It's for stakeholders to understand the "why" and the "what."

- **Executive Summary:** A high-level brief of what the system does.
- **Objectives:** (e.g., "Reduce stockouts by 20%" or "Automate reorder points").
- **Tech Stack:** The languages, databases (like PostgreSQL or MongoDB), and frameworks used.
- **Key Features:** A bulleted list of core functionalities (Barcode scanning, reporting, etc.).

2. Technical Documentation

This is for the developers. It explains the "engine" under the hood.

- **System Architecture:** A diagram or description of how the frontend, backend, and database interact.
- **Database Schema:** Definitions of tables (Products, Suppliers, Transactions, Users) and their relationships.
- **API Documentation:** If the system connects to other tools, list the endpoints and request/response formats.
- **Installation Guide:** Step-by-step instructions on how to set up the environment and run the code.

3. Functional Requirements & Logic

This is the most critical part—it defines the "rules" of your inventory.

- **User Roles & Permissions:** Who can view stock vs. who can delete records (Admin, Manager, Staff).
- **Inventory Logic:** * **Valuation Method:** Are you using FIFO (First-In, First-Out), LIFO, or Weighted Average?
    - **Reorder Logic:** The mathematical formula used to trigger alerts.

  For example, a simple reorder point R can be calculated as:

  $$R = (d \times L) + s$$

  Where d is average daily usage, L is lead time, and s is safety stock.

- **Workflow Diagrams:** Flowcharts showing how an item moves from "Received" to "Sold."

4. User Manual & Maintenance

This is for the daily users (the warehouse team).

- **Getting Started:** How to log in and navigate the dashboard.

- **Common Tasks:** "How to add a new SKU," "How to generate a monthly report."
- **Troubleshooting:** A FAQ for common errors (e.g., "Duplicate SKU detected").
- **Data Backup & Security:** How the data is saved and kept safe.

- ## Project Title:
- Hayakawa Jig IMS Development
- ## Reporting Period:
- Initial Phase / Current Architecture
- ## Tech Stack:
- PHP, JavaScript, CSS, HTML, MySQL (XAMPP)
- ## 1. Project Overview and Problem Statement:
- The Jig Department currently faces challenges in maintaining accurate records of specialized tools and materials.
  The absence of a dedicated digital system has led to manual tracking, increasing the risk of stock inaccuracies and limiting data accessibility.
  The objective of this project is to develop a lightweight, high-performance web-based Inventory Management System (IMS)
  to centralize inventory data, support dynamic database schema updates, and provide a user-friendly interface for department staff.
- ## 2. Current System Architecture:
- The system is implemented using a standard WAMP environment through XAMPP. It incorporates a dynamic database handler
  that enables the application to automatically adapt when new columns are added to the MySQL database without requiring manual changes to the frontend code.

  Key architectural components include:
  • Database Integration – Connection to a MySQL database named 'ims_db' via a centralized configuration file.
  • Dynamic UI Rendering – The frontend retrieves column names directly from the database schema using PHP to ensure the interface reflects the current structure.
  • AJAX Synchronization – A 'Sync Changes' feature allows bulk updates through a modal interface without requiring a full page reload.
-
- ## 3. Progress Summary and Implemented Functionality:
- • Database Connectivity – A functional database connection script has been implemented with error handling.

• Inventory Display – The system dynamically renders inventory data, including category, item name, description, quantity, and price.
• Automated Calculations – A 'Total' column is automatically computed as quantity multiplied by price.
• Addition Logic – New entries are inserted using duplicate-key logic to increment quantity when an item already exists.
• Bulk Management – Users can edit multiple rows simultaneously and delete selected entries.
• Dynamic Column Creation – The backend automatically detects and creates new database columns when required.

- ## 4. System Operational Flow:
- 1. The user opens the inventory page through a web browser.
  2. PHP retrieves inventory records and column metadata using SELECT and SHOW COLUMNS queries.
  3. The system dynamically builds the inventory table interface.
  4. The user selects 'Edit' to open a modal containing an editable copy of the table.
  5. The user modifies values directly within editable cells.
  6. Upon selecting 'Sync Changes', the updated data is sent to the server using AJAX in JSON format.
  7. The backend validates columns, sanitizes inputs, and updates the MySQL database accordingly.
  8. The interface refreshes to reflect the latest data.
- ## 5. System Logic Overview (Pseudo-Code):
- Frontend Logic (JavaScript):
  • Open modal when 'Edit' is clicked.
  • Clone inventory table into modal view.
  • Enable editable mode for allowed cells.
  • Collect edited data and convert to JSON.
  • Send POST request to backend.
  • Refresh interface after successful update.

  Backend Logic (PHP):
  • Verify column existence and create if needed.
  • Insert new items using duplicate-key logic.
  • Iterate through JSON update data.
  • Sanitize and validate inputs.
  • Execute UPDATE queries for each modified row.
- ## 6. Planned Improvements and Next Steps:
- • History Logging – Implement a logging system to track all inventory modifications.
  • Reporting Tools – Add export features such as PDF generation and Excel export.
  • User Authentication – Implement a login using password hashing and session management.
  • Access Control – Restrict editing privileges based on user roles.
-